LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL [3] – [Konsep Dasar Pemrograman Berorientasi Objek]



NAMA : Abdullah Ahmad Izzah

NIM : 103132430024

KELAS : S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek atau Object Oriented Programming (OOP) merupakan metode pemrograman yang membangun sistem dengan komponen yang dienkapsulasi menjadi kelompok data dan fungsi. Komponen-komponen tersebut dapat memiliki hubungan pewarisan serta dapat saling berinteraksi satu sama lain. Dalam pemrograman berorientasi objek terdapat beberapa karakteristik utama, yaitu abstraksi, enkapsulasi, pewarisan, polymorphism, dan komunikasi antar objek atau message.

B. Guided

1. [Abstraksi]

File Abstraksi.java

```
[package guided;

public class Abstraksi {
    public static void main(String[] args) {
        Kucing aren = new Kucing();

        aren.nama = "Aren Gentong ";
        aren.ras = "Mujaer ";
        aren.bersuara();
    }
}]
```

File Kucing.java

```
[package guided;

public class Kucing {
    String nama;
    String ras;

    void bersuara(){
        System.out.println(nama + ras + "meow");
    }
}]
```

Output :

```
Aren Gentong Mujaer meow
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
```

2. [Enkapsulasi]

File Rekening.java
[package guided_enkapsulasi;

```
public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah){
        saldo = saldo + jumlah;
        System.out.println("Saldo Berhasil ditambahkan");
    }

    public void tampilkanSaldo(){
        System.out.println("Saldo Anda : " + saldo);
    }
}]
```

File Enkapsulasi.java
[package guided_enkapsulasi;

```
public class Enkapsulasi {

    public static void main(String[] args){
        Rekening rekGua = new Rekening();

        rekGua.tambahSaldo(150000000);
        rekGua.tampilkanSaldo();
    }

}]
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Project_
Saldo Berhasil ditambahkan
Saldo Anda : 150000000
------------------------------------------
BUILD SUCCESS
------------------------------------------
```

C. Unguided

1. Soal Unguided 1

[Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya.
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Program ini harus memuat dua jenis field: **instance field** (milik masing-masing object) dan **class field** (milik class, dipakai bersama).
- Tidak perlu inheritance, overriding, maupun abstract class.]

Jawaban : file PengolahSuhu.java

```
[package unguided;

public class PengolahSuhu {

    // Instance field: setiap object memiliki array suhu masing-masing
    private double[] suhuHarian;
    // Class field: nilainya tetap dan digunakan bersama oleh semua object
    private static final double NILAI_KOSONG = -1.0;
    // Constructor // Nama parameter sama dengan nama field, sehingga menggunakan this
    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian;
    }

    // Menampilkan seluruh data suhu
    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println("Hari " + (i + 1) + " : " + suhuHarian[i] + "°C");
            }
        }
    }

    // Mencari index yang memiliki nilai kosong
    public int cariIndexKosong() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                return i;
            }
        }
```

```
        return -1;
    }

    // Mengisi data yang kosong berdasarkan rata-rata
    // suhu satu hari sebelum dan satu hari sesudahnya
    public void isiDataKosong() {
        int indexKosong = cariIndexKosong();
        if (indexKosong != -1) {
            suhuHarian[indexKosong] = (suhuHarian[indexKosong - 1] +
suhuHarian[indexKosong + 1]) / 2;
        }
    }

    // Menghitung rata-rata suhu
    public double hitungRataRata() {
        double total = 0;
        for (int i = 0; i < suhuHarian.length; i++) {
            total += suhuHarian[i];
        }
        return total / suhuHarian.length;
    }
}
```

file Main.java

```
[package unguided;

import java.util.Arrays;

public class Main {

    public static void main(String[] args) {

        double[] suhuHarian = {30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9};

        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

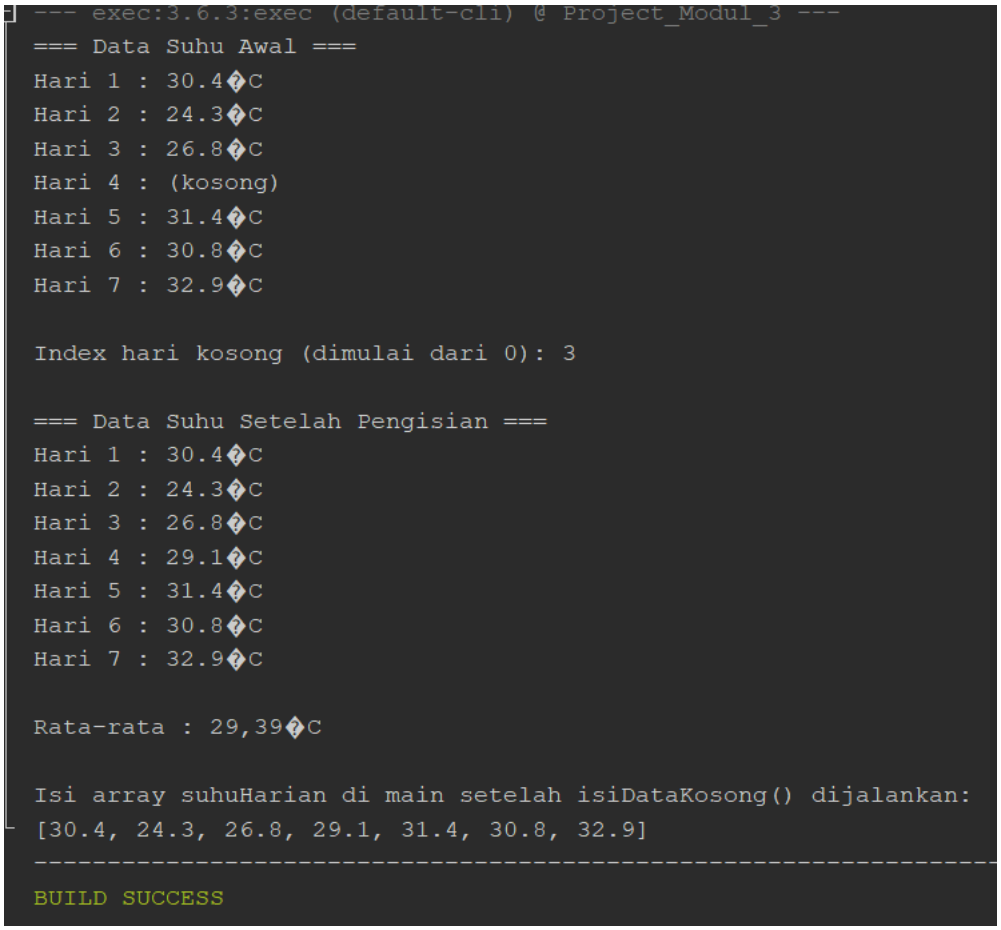
        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();

        int indexKosong = pengolah.cariIndexKosong();

        System.out.println();
        System.out.println("Index hari kosong (dimulai dari 0): " + indexKosong);
        pengolah.isiDataKosong();
        System.out.println();
        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
        System.out.printf("%nRata-rata : %.2f°C%n", pengolah.hitungRataRata());
        System.out.println();
```

```
        System.out.println("Isi array suhuHarian di main setelah " + "isiDataKosong()
dijalankan:");
        System.out.println(Arrays.toString(suhuHarian));
    }
}
```

Output :



Penjelasan :

Program tersebut terdiri dari class PengolahSuhu dan class Main. Pada class PengolahSuhu, terdapat instance field suhuHarian bertipe double[] yang digunakan untuk menyimpan data suhu selama tujuh hari. Selain itu, terdapat class field NILAI_KOSONG yang menggunakan static final dengan nilai -1.0 sebagai penanda data yang kosong. Constructor PengolahSuhu menerima array suhu dari main dan menggunakan keyword this untuk menghubungkan parameter dengan field milik object. Method tampilkanData() digunakan untuk menampilkan seluruh data suhu dan memberikan keterangan (kosong) apabila nilai pada suatu index sama dengan NILAI_KOSONG.

Method cariIndexKosong() digunakan untuk mencari posisi data yang memiliki nilai kosong dengan melakukan perulangan pada seluruh isi array. Method tersebut

mengembalikan index data yang kosong dan mengembalikan nilai -1 apabila tidak ditemukan data kosong. Setelah index ditemukan, method isiDataKosong() akan memanggil cariIndexKosong() kemudian menghitung nilai suhu pengganti menggunakan rata-rata suhu pada satu hari sebelum dan satu hari sesudahnya. Pada data yang digunakan, nilai pada index 3 diisi dengan (26.8 + 31.4) / 2 sehingga menghasilkan suhu 29.1°C. Selanjutnya, method hitungRataRata() digunakan untuk menjumlahkan seluruh data suhu dan membaginya dengan jumlah data sehingga diperoleh rata-rata suhu sebesar 29.39°C.

Pada class Main, array suhu disiapkan terlebih dahulu sesuai data yang diberikan, kemudian array tersebut dikirimkan ke constructor saat object PengolahSuhu dibuat. Program selanjutnya memanggil method untuk menampilkan data awal, mencari index yang kosong, mengisi data kosong, dan menghitung rata-rata suhu setelah data diperbaiki. Setelah itu, array suhuHarian yang berada di main ditampilkan kembali menggunakan Arrays.toString(). Array tersebut ikut mengalami perubahan karena array merupakan tipe reference dan constructor menyimpan referensi ke array yang sama, bukan membuat salinan baru. Oleh karena itu, ketika method isiDataKosong() mengubah isi array melalui object PengolahSuhu, perubahan tersebut juga terlihat pada array yang berada di main.

.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep dasar Pemrograman Berorientasi Objek dapat diterapkan untuk mengolah data suhu menggunakan class, object, field, constructor, dan method. Program berhasil menyimpan data suhu harian dalam instance field, menggunakan class field NILAI_KOSONG sebagai penanda data kosong, serta memanfaatkan keyword this pada constructor. Data suhu yang kosong pada Hari 4 berhasil ditemukan berdasarkan index dan diisi menggunakan rata-rata suhu pada hari sebelum dan sesudahnya. Selain itu, hasil praktikum menunjukkan bahwa array yang dikirim ke constructor merupakan tipe reference, sehingga perubahan data yang dilakukan melalui object PengolahSuhu juga menyebabkan array yang berada di main ikut berubah. Dengan demikian, praktikum ini membantu memahami penerapan konsep class, object, method, constructor, keyword this, dan penggunaan reference pada Java dalam pengolahan data sederhana.